

JavaScript Closure Interview Questions

JavaScript Interview Preparation Guide

Q1. Closure কী? (এক লাইনে)

Ans: Closure হলো এমন একটি feature যেখানে একটি inner function তার outer (lexical) scope-এর variables মনে রাখতে পারে এবং outer function শেষ হওয়ার পরেও সেই variables access করতে পারে।

Q2. Closure আর Normal Function এর পার্থক্য কী?

Ans:

Normal Function:

- Outer function শেষ হলে তার local variables নষ্ট হয়ে যায়।
- Local variables পরে access করা যায় না।

Closure:

- Inner function outer function-এর variables মনে রাখে।
- Outer function শেষ হলেও সেই variables ব্যবহার করতে পারে।

Example:

```
function createCounter() {  
    let count = 0;  
  
    return function () {  
        return ++count;  
    };  
}  
  
const counter = createCounter();  
  
counter(); // 1  
counter(); // 2  
counter(); // 3
```

এখানে `count` variable-টি `createCounter()` শেষ হওয়ার পরেও মনে রাখা হয়েছে। এটাই **Closure**।

Q3. Closure দিয়ে private variable কীভাবে বানাবেন?

Ans: Closure ব্যবহার করে variable-কে বাইরের code থেকে hide (private) রাখা যায়।

Example:

```
function cashCounter() {  
    let amount = 0; // private variable  
  
    return function(pay = 0){  
        amount += pay;  
        return amount;  
    };  
}  
  
const shop = cashCounter();  
  
shop(100); // 100  
shop(50);  // 150  
  
console.log(amount);
```

Output: ReferenceError: amount is not defined

কারণ: amount শুধুমাত্র cashCounter() এর ভিতরে থাকে। এটি বাইরের code থেকে সরাসরি access করা যায় না।

Q4. Loop এ var ব্যবহার করলে setTimeout ভুল output দেয় কেন?

Ans: কারণ **var** হলো function-scoped। Loop শেষ হওয়ার পর সব callback একই variable share করে।

Example (Problem):

```
for(var i = 1; i ≤ 3; i++){  
  setTimeout(()⇒{  
    console.log(i);  
  }, 1000);  
}
```

Output: 4, 4, 4

কারণ: Loop শেষ হওয়ার পর **i = 4** হয়ে যায়। সব callback একই **i** variable রেফার করে।

সমাধান: **let** ব্যবহার করলে প্রতিটি iteration-এর জন্য নতুন block-scope variable তৈরি হয়।

```
for(let i = 1; i ≤ 3; i++){  
  setTimeout(()⇒{  
    console.log(i);  
  }, 1000);  
}
```

Output: 1, 2, 3

Q5. Closure কি Memory Leak তৈরি করতে পারে? কেন?

Ans: হ্যাঁ, পারে।

কারণ Closure outer function-এর variables memory-তে ধরে রাখে যতক্ষণ inner function-এর reference বেঁচে থাকে। যদি অনেক বড় object বা unnecessary data closure-এর ভিতরে ধরে রাখা হয় এবং reference remove না করা হয়, তাহলে Garbage Collector সেটি memory থেকে release করতে পারে না। এতে memory leak হওয়ার সম্ভাবনা থাকে।

নোট: সাধারণভাবে Closure নিজে কোনো memory leak নয়। কিন্তু অসাবধানতাবশত অপ্রয়োজনীয় reference ধরে রাখলে memory leak হতে পারে।